

SevaTrack: Grievance Redressal Portal - Project Explanation Document

This document is prepared as a comprehensive guide for your project review/viva. It covers all the aspects requested, tailored for a college presentation.

1. PROJECT OVERVIEW

What the application does: SevaTrack is a comprehensive Grievance Redressal Portal designed to streamline the process of logging, tracking, and resolving civic or organizational issues. It allows users to register complaints, track their status in real-time, and enables authorities to manage and resolve these issues efficiently through a dedicated dashboard.

What problem it solves: Traditionally, filing a grievance involves physical paperwork, long queues, and a lack of transparency regarding the status of the complaint. Users often feel their voices are unheard. SevaTrack digitizes this process, ensuring accountability, transparency, and faster resolution of problems.

Target users:

- **Citizens/End Users:** Individuals who need to report issues (e.g., civic problems like potholes, water supply issues, or organizational grievances).
- **Administrators/Authorities:** Officials responsible for reviewing, assigning, and resolving the reported grievances.

Why this problem needs a solution: A lack of an efficient grievance redressal system leads to public dissatisfaction and administrative inefficiency. A digital platform bridges the communication gap between users and authorities, fostering trust and ensuring structured problem-solving.

Real-world use cases:

- A citizen reporting a broken streetlight in their neighborhood.
- A student raising a maintenance issue in a university hostel.
- An employee reporting a workplace facility issue to the HR/Administration department.

2. COMPLETE APPLICATION ARCHITECTURE

Full Flow: User → Frontend (Next.js/React) → Authentication → Backend Services (Next.js API Routes/Supabase) → Database → Response

End-to-End Workflow:

- **How a user opens the app:** The user navigates to the web application URL. The frontend, hosted on Vercel/GitHub Pages, serves the initial HTML, CSS, and JavaScript.
 - **How login/signup works:** The user enters credentials on the login/signup page. The frontend sends these credentials to the Authentication service. Upon successful verification, a secure session token is generated and stored locally.
 - **How data is stored:** When a user submits a grievance, the frontend sends a POST request with the grievance details (title, description, category, images) to the backend API. The backend validates the data and inserts a new record into the database.
 - **How data is retrieved:** When the user visits their dashboard, the frontend makes a GET request to the backend. The backend queries the database for grievances linked to the user's ID and returns the data as JSON, which the frontend renders.
 - **How updates happen:** If an administrator updates the status of a grievance (e.g., from "Pending" to "Resolved"), the frontend sends a PATCH/PUT request to the backend. The backend updates the specific record in the database, and the UI reflects this change.
 - **How security is maintained:** Security is maintained through secure token-based authentication, Role-Based Access Control (RBAC) to ensure users only see their data and admins see all data, and secure API endpoints that validate every request.
-

3. FRONTEND EXPLANATION

Frontend Technology Used: React.js framework via Next.js. **Programming**

Languages: TypeScript / JavaScript, HTML, CSS. **Frameworks/Libraries Used:** Next.js, Tailwind CSS (for styling), Radix UI (for accessible UI components), Lucide React (for icons), React Hook Form (for form handling). **UI Components:** Reusable modular components like Buttons, Cards, Dialogs, and Data Tables. **Page Structure:** Single Page Application (SPA) feel with Next.js App Router for optimized routing. **User Interaction Flow:** Landing Page -> Authentication -> User Dashboard -> Submit Grievance Form -> Track Status.

Major Feature Explanation:

Feature: Grievance Submission Form

- **How it works:** Uses react-hook-form to capture user input (category, description, location). It validates the data on the client side before sending an API request to the backend.

- **Why this approach was chosen:** react-hook-form minimizes re-renders, making the form highly performant. Client-side validation provides immediate feedback to the user, improving User Experience (UX).
 - **Comparison with traditional approaches:** Traditional forms submit data by reloading the entire page and relying solely on server-side validation. Our approach is asynchronous (AJAX/Fetch), providing a seamless, no-reload experience.
-

4. BACKEND EXPLANATION

Backend Architecture: Serverless architecture using Next.js API Routes and Cloud Database (Supabase/BaaS). **APIs/Services Used:** RESTful API principles. **Server-side logic:** Handles request validation, authentication checks, business logic (e.g., assigning a grievance ID), and database interactions. **Data processing:** Sanitizing inputs to prevent injection attacks and formatting data before database insertion. **Communication:** Uses standard HTTP methods (GET, POST, PUT, DELETE) transmitting JSON payloads.

Why this backend approach instead of:

- **PHP + MySQL:** PHP is older, synchronous, and often requires manual server configuration (LAMP stack). Our serverless Next.js approach allows writing both frontend and backend in one language (TypeScript) and auto-scales without server management.
- **Traditional server-based backend (e.g., Express/Node.js on EC2):** A traditional server requires continuous maintenance, patching, and scaling. Serverless functions only run when requested, reducing costs and maintenance overhead.
- **Custom backend development from scratch:** Using a Backend-as-a-Service (BaaS) like Firebase/Supabase drastically reduces development time by providing pre-built authentication, database APIs, and storage, allowing focus on core features.

Advantages: High scalability, unified codebase, rapid development. *Disadvantages:* Vendor lock-in, less control over the underlying infrastructure compared to a custom server.

5. AUTHENTICATION SYSTEM

Complete Authentication Flow:

- **User registration:** User provides email and password. The system hashes the password and creates a new user profile.
- **Login:** User enters credentials. The system verifies the hash.

- **Session/token management:** Upon successful login, a JSON Web Token (JWT) or secure session cookie is issued. This token is sent with subsequent requests to prove the user's identity.
- **User verification:** API routes check the validity of the token before granting access to protected data.

Comparison:

- **Basic username/password authentication:** Basic auth sends credentials with *every* request, which is highly insecure if not over HTTPS, and inefficient.
- **Manual authentication systems:** Building auth from scratch involves managing password hashing (bcrypt), salting, session storage, and security vulnerabilities. Our selected system handles these complexities securely out-of-the-box.

Why the selected authentication system is better: It adheres to modern security standards, supports secure token lifecycles, prevents brute-force attacks, and is maintained by security experts rather than relying on custom, potentially flawed logic.

6. DATABASE EXPLANATION

Database Technology Used: Cloud-based Database (Supabase PostgreSQL / Firebase concepts applied). **Database Structure:** Relational/Document structure based on the service used. **Collections/Tables:**

- **Users:** Stores user details and roles (admin/citizen).
- **Grievances:** Stores complaint details, status, timestamps, and the reporting user's ID. **Data relationships:** A One-to-Many relationship where one User can have multiple Grievances. **CRUD operations:** Create (submit grievance), Read (view dashboard), Update (change status), Delete (remove duplicate complaints).

Firestore/NoSQL vs SQL (MySQL):

- **NoSQL (Firestore):** Stores data in flexible, JSON-like documents. Great for rapid iteration, hierarchical data, and real-time synchronization. Schema-less.
- **SQL (MySQL):** Stores data in rigid tables with strict schemas and complex joins. Better for highly structured data with complex relationships.

Why Firestore (or a BaaS approach) was selected:

- **Scalability:** Automatically scales to handle thousands of concurrent users without manual database tuning.

- **Real-time updates:** Built-in WebSocket support allows the dashboard to update instantly when a grievance status changes.
 - **Faster development:** No need to write complex SQL queries or manage database migrations initially.
 - **Security rules:** Allows writing declarative security rules to control data access directly at the database level.
 - **Cloud support:** Fully managed, meaning no database servers to patch or back up.
 - **Less backend maintenance:** Focus remains on the frontend user experience rather than database administration.
-

7. COMPLETE FEATURE ANALYSIS

1. Feature name: User Authentication

- **Purpose:** Secure the application and ensure users only see their own data.
- **How it works technically:** Token-based verification on route access.
- **Technology used:** JWT, BaaS Auth.
- **User benefit:** Data privacy and secure identity management.
- **Existing solutions:** Traditional session cookies.
- **Why my implementation is better:** Stateless and easily scalable across different platforms (web, mobile).

2. Feature name: Real-time Dashboard

- **Purpose:** Allow users to see the current status of their grievances.
- **How it works technically:** Fetches data on load and listens for database changes.
- **Technology used:** React State, Fetch API / Real-time listeners.
- **User benefit:** Immediate transparency without needing to refresh the page.
- **Existing solutions:** Static tables requiring manual refresh.
- **Why my implementation is better:** Dynamic and provides a modern, responsive user experience.

3. Feature name: Status Tracking

- **Purpose:** Show the lifecycle of a complaint (Pending -> In Progress -> Resolved).

- **How it works technically:** Admin updates a status enum in the database, which reflects on the user's UI.
- **Technology used:** REST API PATCH request.
- **User benefit:** Keeps the user informed, reducing anxiety and follow-up inquiries.
- **Existing solutions:** Phone call follow-ups or email threads.
- **Why my implementation is better:** Centralized, automated, and accessible 24/7.

8. TECHNOLOGY JUSTIFICATION

Technology I used	Alternative	Why my choice is better
Next.js (React)	Plain HTML/JS or Angular	Better SEO, Server-Side Rendering capabilities, and a massive ecosystem of reusable components.
Serverless/BaaS	Node.js + Express	Zero server maintenance, auto-scaling, and significantly faster development speed.
Cloud DB (Firebase/Supabase)	Local MySQL	Built-in APIs, real-time capabilities, and high availability without manual setup.
Token Auth	Custom Session Auth	More secure, stateless, and integrates easily with modern JavaScript frameworks.
Vercel/Cloud Hosting	Traditional cPanel	Automated CI/CD deployments directly from GitHub, global CDN for fast loading.
Tailwind CSS	Custom Vanilla CSS	Utility-first approach allows rapid UI building without leaving the HTML file, ensuring a consistent design system.

9. SECURITY ANALYSIS

- **Authentication security:** Passwords are never stored in plain text. JWTs are used for secure session management, preventing unauthorized access.
- **Database security rules:** Row Level Security (RLS) or specific database rules ensure that a user can only read and write data that belongs to their specific User ID.

- **User data protection:** Data is transmitted over HTTPS, encrypting communication between the client and server.
 - **Possible attacks prevented:**
 - *SQL Injection/NoSQL Injection:* Prevented by using parameterized queries and ORM/SDKs provided by the BaaS.
 - *Cross-Site Scripting (XSS):* React automatically escapes variables in JSX, preventing malicious scripts from being injected into the UI.
 - *Cross-Site Request Forgery (CSRF):* Mitigated by using secure tokens and modern browser CORS policies.
-

10. WHY THIS PROJECT IS BETTER THAN EXISTING APPLICATIONS

Unique features: A highly intuitive, modern UI focused on accessibility, unlike clunky government or legacy enterprise portals. **Improvements:** Replaces manual paperwork and fragmented email chains with a centralized, digital dashboard. **User experience advantages:** Instant feedback, mobile-responsive design, and clear status indicators make it extremely easy for non-technical users to file a complaint. **Technical advantages:** Built on a modern serverless stack, making it highly performant, scalable, and easy to maintain compared to monolithic legacy applications.

11. FUTURE IMPROVEMENTS

- **New features:** Multi-language support for broader accessibility, and an admin analytics dashboard to track grievance resolution times.
 - **Scaling improvements:** Implementing caching (e.g., Redis) for frequently accessed data to reduce database load.
 - **AI integration possibilities:** Integrating an AI chatbot to help users categorize their grievances automatically, or using sentiment analysis on grievance descriptions to prioritize urgent issues.
 - **Performance improvements:** Image compression before uploading to save storage space and bandwidth.
-

12. FINAL PRESENTATION SUMMARY (To Speak in Viva)

"Good morning respected teachers. I have built **SevaTrack**, a modern Grievance Redressal Portal.

Why I built it: I built this because traditional methods of filing complaints are slow, lack transparency, and involve too much paperwork. I wanted to create a platform where users can easily report issues and track them in real-time.

Technologies used: I used **Next.js and React** for the frontend to create a fast, interactive user interface. For the backend and database, I utilized a **Cloud Serverless architecture (BaaS)** because it provides secure authentication and a scalable database out-of-the-box. I styled the application using **Tailwind CSS**.

Why these technologies were chosen: I chose React/Next.js over older technologies like PHP because it allows for a seamless, single-page application experience. I chose a cloud database over a traditional SQL server because of its real-time capabilities and zero maintenance overhead, allowing me to focus on building features.

Main advantages: The main advantages of my project are its intuitive user experience, real-time status tracking, and robust security. It replaces manual, frustrating processes with an automated, transparent digital system.

Thank you. I am open to any questions."